

Recursive Brute Force Solutions

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Brute Force

- Straightforward approach to solving a problem that is usually based on the problem statement
- Systematically checks all possible candidates for a solution
- It is exhaustive and typically, no optimization is involved

General Framework /1

- Enumerate all possible solutions
- For each candidate, check if it is a solution

General Framework /2

```
void solve(int *solution, int size, int pos) {  
  
    // Base Case is when the solution array  
    // is completely filled  
    if( pos == size ) {  
        // Candidate solution is found  
        // So check if it is a solution or not  
        check_solution( solution, size );  
  
        return;  
    }  
  
    // Fill the current slot (e.g., 0)  
    //solution[pos] = 0;  
  
    // Recursive Case is to simply fill  
    // the remaining slots  
    solve( solution, size, pos+1 );  
  
}
```

Preliminary

Given an integer **N**, enumerate all binary numbers of length **N**.

For example, if **N=4**: 0000 to 1111

$\begin{array}{cccc} _ & _ & _ & _ \end{array}$

of choices $2 * 2 * 2 * 2 = 2^4$ or 16 possibilities from 0000 to 1111

Problem Statement

Given a **N**-digit odometer (e.g., 4 slots), return how many numbers contain the digit 8 at least three times.

— — — —

Enumeration

There are **37 numbers** that contains the digit 8 at least three times.

0	8	8	8
1	8	8	8
2	8	8	8
.	.	.	.
8	9	8	8
9	8	8	8

Discussion

- It seems that you just have a for loop for each slot
- However, the length of the number (i.e., slots) may change
- That is why it is easier to implement this **recursively**
- There are certain cases where we could have stopped earlier*

Problem Statement

Given an array of N distinct positive integers, return the number of subsets whose sum is an odd number.

3, 6, 7, 9

Subset Generation

There are **8 subsets** where the sum is an odd number.

{ 9 }

{ 7 }

{ 6 9 }

{ 6 7 }

{ 3 }

{ 3 7 9 }

{ 3 6 }

{ 3 6 7 9 }

Discussion

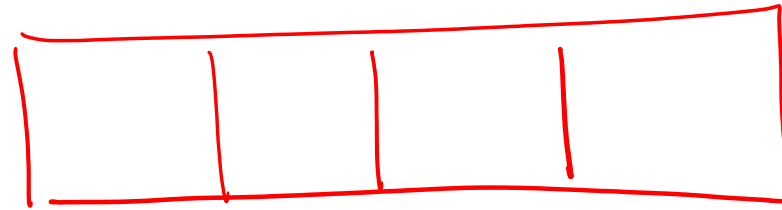
- Approach was similar to the odometer problem
- However, here, we have a different list of numbers we want to test
- Look at this problem in the perspective of binary numbers

Problem Statement

Given a list of **N** distinct symbols, list all permutations.

3, 6, 7, 9

of possibilities



4 * 3 * 2 * 1 = 4! or 24 possibilities

Permutation

There are **24** possible permutations.

3 6 7 9

3 6 9 7

3 7 6 9

• • • •

9 6 7 3

9 7 3 6

9 7 6 3

Discussion

- We generated all the possible arrangements of all the elements
- Therefore, we must keep track which elements have been used

Final Notes

- We have seen various brute force solutions
- Simple to formulate but the tradeoff is the slow running time
- They come in many forms, but has the same general idea

Questions?